Product Development Document: Restaurant Order & Kitchen Display System

Working product name: **KitchenFlow** · Version 0.1 (draft) · 27 October 2026 · Author: Rathi · Stack: Java, Spring Boot, SQL

1. Purpose of this document

This document answers one question: what are we building, for whom, and why, before any design or code. It fixes the scope so the next six documents have a stable target. It is deliberately light on technology detail; that comes in the design documents.

Proposed document set

#	Document	What it answers
1	Product Development Document (this one)	What and why, scope, roadmap
2	Software Requirements Specification (SRS)	Numbered functional and non-functional requirements
3	System Architecture (HLD)	Modules, components, real-time flow, deployment
4	Database Design	ERD, tables, constraints, indexes, locking strategy
5	API Specification	Endpoints, request/response, errors, roles
6	Low-Level Design	Classes, state machines, sequence diagrams
7	Test Plan and Test Cases	Strategy, concurrency and failure cases, acceptance tests

Rule: documents 1 to 4 come before any code. Documents 5 to 7 are written just ahead of the build phase that needs them. Keep each document to a few pages; a short document that matches the code beats a long one that does not.

2. Problem and vision

Quick service restaurants (QSRs) such as KFC, McDonald's and Burger King run on speed. A rush-hour order passes through the counter, the kitchen stations (fryer, grill, assembly, beverages) and the hand-over point, and every handoff is a place where orders get lost, delayed or duplicated. Paper KOT (kitchen order ticket) printers and shouting do not scale at peak hours: nobody can see which order is late, which item is out of stock, or how long preparation really takes.

Vision: a backend-first system that takes an order at the counter, splits it automatically into station-level tickets, shows them live on kitchen screens, tracks each order until hand-over, and gives the manager timing and sales data afterwards.

One-line pitch: an order-to-kitchen pipeline for a QSR outlet that takes the order, routes it to the right station, tracks it in real time, and measures how fast the kitchen really is.

3. Domain background: how a QSR actually works

Know these terms well enough to speak about them without notes.

- **Order channels:** counter POS, self-order kiosk, mobile or pickup app, delivery aggregators. Version 1 builds the counter channel, plus an optional simulated external channel.
- **Order types:** dine-in, takeaway, delivery.
- **Token number:** the customer gets a short token and watches a status board (Preparing or Ready).
- **Station routing:** each menu item belongs to a station, for example fries to the fry station, burgers to grill and assembly, drinks to beverages.
- **Combos:** one bill line (burger, fries, drink) becomes several station tasks.
- **Modifiers:** no onion, extra cheese, spicy. They must reach the cook accurately.
- **KDS (Kitchen Display System):** the screen that replaces paper KOT. Cooks start a ticket and bump it (mark it done) when finished; a recall undoes a bump.
- **Expo (expeditor):** checks that every part of an order is ready and hands it over together.
- **86ing:** marking an item unavailable when stock runs out, so the cashier cannot sell it.
- **Timers and SLA:** each ticket ages, and its colour (green, amber, red) shows how far it is past the target preparation time.

- **Rush hours:** lunch and dinner spikes, with many orders and many staff taps at the same moment. This is where correctness problems appear.
- **Tax:** GST on the bill, configured as a rate per item category.

4. Goals and non-goals

Goals

1. Correctness under rush: no lost, duplicated or double-updated orders.
2. Real-time visibility: a new order appears on the right station screen within about a second.
3. A realistic order lifecycle with enforced state transitions and an audit trail.
4. Measurable kitchen performance: preparation time per station, late orders, top items.
5. Easy to run and demonstrate: one command to start, seeded data, a 5-minute demo.
6. Defensible engineering: every design choice can be explained in an interview.

Non-goals for version 1: real payment gateway, real aggregator integrations (Swiggy, Zomato), multi-outlet franchise management or multi-tenant SaaS, printer or POS hardware integration, demand forecasting, microservices or Kafka. Reason: one developer and limited time, and depth in a few things beats breadth.

5. Users and roles

Role	Who	Main needs
Cashier	Takes orders at the counter	Fast order entry, see item availability, record payment, cancel before preparation
Kitchen staff	Cook at a station	See only their station's tickets; start, bump, recall
Manager / Expeditor	Oversees the flow	See all orders and delays, hand over, mark rush, approve late cancellations, 86 items
Admin / Owner	Configures the outlet	Manage menu, stations, prices, tax and staff accounts; view reports
Customer (indirect)	Waits for the order	Token and status board, read-only, no login

6. Product scope (MoSCoW)

Must (the MVP the demo needs)

1. Login with roles: Cashier, Kitchen, Manager, Admin.
2. Menu: categories, items, modifiers, price, station mapping, availability toggle.
3. Order entry: multiple items, modifiers, order type, token number generation.
4. Order lifecycle with enforced transitions and cancellation rules.
5. Automatic routing of order items into station tickets.
6. KDS per station: live tickets, start, bump, recall, age timer.
7. Customer status board: token with Preparing or Ready.
8. Payment record (cash, UPI or card as simulated statuses) and a bill with GST.
9. Order history with search, filters and pagination.
10. Daily sales and kitchen timing summary.

Should (what makes an interviewer lean forward)

- Combo items.
- Inventory-lite with automatic 86 at zero stock.
- Idempotent order creation.
- Audit log of every state change.
- Rush priority flag and late-order alerts.

Could (only after everything else is solid)

- Simulated delivery-aggregator webhook, receipt PDF, an Android KDS client, an outlet id field for future multi-outlet use.

Won't (this version): everything under non-goals above.

7. Core user journeys

A. Counter order (happy path). The cashier builds an order, the system validates items and availability, the order is created with a token, payment is recorded, and the order is accepted. It is routed into station tickets, the KDS shows them, cooks start and bump them, and when every ticket is done the order becomes READY. The status board shows the token, the order is handed over, and it is COMPLETED.

B. Cancellation. Before preparation starts, the cashier can cancel and the tickets are removed. After preparation starts, a manager must approve, a reason is recorded, and the order is flagged as wastage.

C. Item runs out. The manager marks an item as 86. The menu shows it unavailable and the cashier cannot add it; orders already placed are not affected.

D. Kitchen mistake. A cook bumps the wrong ticket and recalls it. The ticket returns to in progress, and both actions are audited.

E. Rush hour. Thirty or more orders arrive within a minute. There are no duplicates, tickets are ordered by arrival and priority, and every KDS screen stays in sync.

State models

- Order: CREATED → PAID → IN_KITCHEN → READY → COMPLETED. CANCELLED is reachable from CREATED, PAID and IN_KITCHEN under the rules in journey B.
- Ticket: NEW → IN_PROGRESS → DONE. A recall moves DONE back to IN_PROGRESS.
- An order becomes READY only when all of its tickets are DONE.

8. Functional modules (overview)

Module	Responsibility	Key concepts
Auth and Users	Login, roles, access rules	JWT, role-based authorization
Menu	Items, categories, modifiers, station mapping, availability	Validation, soft delete
Orders	Creation, pricing, tax, lifecycle	State machine, idempotency
Kitchen (KDS)	Tickets, station queues, bump and recall, timers	Real-time push, optimistic locking
Payments	Simulated payment records and statuses	Idempotent payment recording
Inventory-lite	Stock counters, automatic 86	Atomic updates
Reporting	Sales and preparation-time metrics	SQL aggregation
Common	Error handling, audit log, configuration	One consistent error format

9. Quality targets (non-functional)

These are targets to be validated by tests, not guesses.

- **Correctness:** retrying the same order request creates exactly one order. Two cooks acting on one ticket never corrupt its state.
- **Performance:** order placement p95 under 300 ms with 50 concurrent users on a laptop; a new ticket visible on the KDS within 1 second.
- **Security:** BCrypt-hashed passwords, role checks on every endpoint, no sensitive data in logs.
- **Reliability:** every state change inside a transaction; the app restarts without data loss; a KDS screen that reconnects reloads the current tickets.
- **Observability:** structured logs carrying the order id, plus health and metrics endpoints.
- **Maintainability:** layered code packaged by feature; at least 70% coverage on the service layer.
- **Operability:** a single docker compose command starts the app and database with seed data.

10. Technical direction (summary; detail in document 3)

Style: modular monolith. One deployable Spring Boot application, packages by feature, clear boundaries between modules. It is easy to run and demo, and you can still explain how modules could become services later.

Area	Default choice	Reason
Language	Java 21 (use 17 if the target company uses 17)	LTS release; records suit DTOs
Framework and build	Spring Boot 3.x with Maven	Most common in enterprise Java
Persistence	Spring Data JPA with PostgreSQL (MySQL is equally fine)	Transactions, locking and constraints; the concepts carry over

Area	Default choice	Reason
Migrations	Flyway. Keep a single baseline migration and replace it wholesale while the schema is still moving; switch to incremental migrations after the first demo build	Fast iteration early, safe history later
Security	Spring Security with JWT	Role-based access
Real-time	Server-Sent Events for KDS push; WebSocket only if two-way messaging is needed	The KDS is one-way push, so SSE is simpler and enough
API docs	springdoc OpenAPI (Swagger UI)	Live, testable documentation
Testing	JUnit 5, Mockito, Testcontainers	Real database in integration tests
Delivery	Docker Compose, GitHub Actions CI	One-command run, automatic checks
Client UI	Lightweight web UI for POS, KDS and status board; framework decided in document 3. An Android KDS client can reuse the same API later	Backend is the focus; UI must be clean, not elaborate

Deliberately excluded: Kafka, Redis, microservices, Kubernetes. Each adds failure modes you would have to explain, and none is needed for one outlet. Present them as the future scaling path, not as part of version 1.

Keep dependencies few. Use plain Java records and straightforward logic rather than heavy abstraction layers.

11. Engineering showcase features

These are what separate this project from a CRUD tutorial.

Feature	Problem it solves	Interview talking point
Order state machine	Illegal transitions, such as cooking a cancelled order	Explicit transition table; a test for every illegal move

Feature	Problem it solves	Interview talking point
Idempotency key on order creation	A double tap or network retry creates duplicate orders	Unique key constraint; the same response is returned on retry
Optimistic locking on tickets	Two staff update the same ticket	Version column; conflict returns HTTP 409
Atomic stock decrement	The last item sold twice during a rush	One conditional UPDATE versus read-modify-write
Real-time push with reconnect	A KDS must never show a stale board	SSE plus a full state reload on reconnect
Audit log	Who did what, and when	Append-only table, reused for timing reports
Concurrency test	Proving the correctness claims	50 threads placing orders at once
Metrics and health	Running the system in practice	Actuator endpoints

12. Roadmap (assumes 8 weeks; adjust to your real deadline)

Phase	Weeks	Output	Done when
0. Planning	1	Documents 1 to 4, repository, CI skeleton	Documents reviewed; no code beyond the skeleton
1. Foundation	2-3	Auth, menu, order placement, state machine (documents 5 and 6 for these)	An order can be created through the API; illegal transitions rejected; tests pass
2. Kitchen	4-5	Routing, tickets, KDS push, bump and recall, status board	An order flows end to end in a live demo
3. Hardening	6	Idempotency, locking, inventory-lite, audit log, concurrency tests (document 7)	The 50-thread test passes with no duplicate orders

Phase	Weeks	Output	Done when
4. Polish	7	UI cleanup, reports, Docker, README, architecture diagram	One-command start; 5-minute demo script ready
5. Interview readiness	8	Pitch, question practice, buffer	You can explain every module without notes

Each phase ends with a working, demoable build. If a phase slips, cut Should items, never the Must list.

13. Success criteria

- A new person can run the project with one command and the README.
- The 5-minute demo works: place orders at the counter, watch tickets appear on two station screens, bump them, see the status board change, and show the sales and timing report.
- All Must features work, and at least four showcase features from section 11 are implemented and tested.
- Every major design decision is recorded in one or two lines in the documents.
- You can answer the questions in section 14 without notes.

14. Interview positioning

Two-minute story: the problem (QSR rush hour), what you built (an order-to-kitchen pipeline), the hardest problem you solved (pick one from section 11), and what you would do next to scale it.

Likely question	Where your answer comes from
Why this project?	QSR is the company's domain; real operational flows, not CRUD
How do you prevent duplicate orders?	Idempotency key
How does the kitchen screen update live?	SSE with reconnect and state reload
Two cooks update one ticket?	Optimistic locking

Likely question	Where your answer comes from
The last burger sold twice?	Atomic stock update
How would you scale to 500 outlets?	Outlet id, stateless app, read replicas, a queue between order and kitchen, menu caching (as a future path)
How did you test it?	Unit tests, Testcontainers integration tests, a concurrency test

Rule: put on your resume only what you can demo and explain line by line, and keep the project's year accurate to when it was actually built.

15. Risks and mitigations

Risk	Impact	Mitigation
Scope creep from tutorials and new ideas	The project never finishes	Freeze the Must list; park new ideas in a backlog
Over-engineering (Kafka, microservices)	You cannot defend it in the interview	Modular monolith; justify every tool in one sentence
Tutorial dependence	Shallow understanding	Design first; use tutorials only for syntax
Time pressure from exams and applications	An incomplete project	Phase exit criteria; cut Should items first
Weak UI	Poor first impression	One consistent, simple layout; cap UI work at one week
Documents drift from code	Confusion later	Update a document whenever the design changes; keep them short

16. Assumptions and open questions

Assumptions: single outlet; English UI; one developer; runs on a laptop, with an optional free cloud demo; simulated payments; GST as a flat configurable rate per category.

Open questions (answer before document 2):

1. When is the first interview or round where you will show this? That sets the real roadmap length.
2. Does the company's job description name a Java version, database (MySQL, PostgreSQL, Oracle) or tools? Match them where it is cheap.
3. UI choice: server-rendered pages, or a small React web app?
4. Does your college or the company define the seven documents? If so, send the list and this set will be mapped to it.

17. Next step

Document 2, the SRS: turn every Must and Should item into numbered requirements with acceptance criteria, using the roles and state models defined here.